

REPORT: STOCK PRICE PREDICTION USING MACHINE LEARNING IN PYTHON

Stock price prediction, also known as stock forecasting, is the process of estimating the future value of a company's stock over a defined period—whether that be daily, weekly, or monthly. It relies on the analysis of historical price data combined with market trends, volume, and economic indicators to produce actionable projections. For investors and financial institutions, stock prediction is a critical component of portfolio management and risk assessment.

Accurate forecasts enable traders to optimize entry and exit points, allocate capital efficiently, and make informed decisions about asset allocation. Poor forecasting, conversely, can lead to significant financial losses and missed investment opportunities. Machine learning (ML) has revolutionized this field by dramatically increasing forecasting accuracy. Unlike traditional statistical techniques, ML algorithms—specifically deep learning models like LSTMs—can automatically identify complex, non-linear patterns hidden within large volumes of temporal data.

To develop an accurate model for stock prediction, a series of steps is followed:

- **Data Collection:** Gathering historical stock data, including Open, High, Low, Close prices, and Volume.
- **Data Preprocessing:** Scaling data (normalization), handling missing values, and creating time-series sequences.
- **Feature Engineering:** Developing features such as Moving Averages (MA), Relative Strength Index (RSI), or lagged returns.

- **Model Selection:** Popular models include Linear Regression for simple trends, Random Forest for complex patterns, and LSTM (Long Short-Term Memory) Networks for sequential time-series prediction.
- **Evaluation:** Measuring model accuracy using metrics like Root Mean Squared Error (RMSE) or Mean Absolute Error (MAE).

Code Analysis

1. Importing Libraries

The code utilizes essential tools for financial data science:

- **yfinance:** To fetch real-time and historical market data.
- **pandas & numpy:** For data manipulation and matrix operations.
- **scikit-learn:** For data scaling (MinMaxScaler).
- **tensorflow/keras:** To build and train the deep learning LSTM architecture.
- **matplotlib:** For creating the final performance visualization.

2. Data Preparation & Scaling

The dataset is fetched for a specific ticker (e.g., 'AAPL'). Because neural networks are sensitive to the scale of input data, a MinMaxScaler is applied to transform prices into a range between 0 and 1. The data is then split, with 80% used for training and 20% reserved for testing.

3. Feature Engineering (Sliding Window)

Machine learning models require a "look-back" period. The code creates sequences where the model looks at the past 60 days of stock prices to predict the price for the 61st day.

4. Training the LSTM Model

The model initializes a Sequential LSTM network. LSTMs are unique because they have "gates" that allow the model to remember important long-term trends while forgetting irrelevant short-term noise. The `model.fit` phase involves the computer iteratively adjusting internal weights to minimize the Mean Squared Error.

5. Prediction and Evaluation

Once trained, the model predicts prices for the "unseen" test period. These predicted values are then "inverse transformed" back from their 0-1 scale to actual USD dollar values for comparison against real market prices.